

1. Logika cyfrowa

Układ	Formuła / sygnatura	Opis
Sumatory		
Półsumator	$s = x \oplus y$ $c = x \& y$	Dodaje dwa bity. s to suma, c to bit przeniesienia (carry).
Pełny sumator (FA)	$s = x \oplus y \oplus ci$ $co = x \& y \mid ci \& (x \oplus y)$	Jak półsumator, ale przyjmuje też przeniesienie wejściowe ci .
Sumator n-bitowy	$n \times \text{FA}$	Kaskada (ripple-carry): wyjście C_i trafia na wejście kolejnego FA. Ostatnie co to Carry Out całości.

Układy kombinacyjne		
Dekoder	$n \rightarrow 2^n$	Wejście koduje liczbę k . Na wyjściu zapalony jest tylko bit nr k .
Multiplekser	$2^n + n \rightarrow 1$	2^n bitów danych + n bitów sterujących S . Na wyjście przechodzi S -ty bit danych.
ALU	$A, B, f \rightarrow C$	Dekoder na bitach f wybiera operację (np. $A + B$, $A \mid B$, $A \& B$); multipleksowanie wyników bramkami AND/OR.

Układy sekwencyjne (pamięć)		
Przerzutnik S-R	S - set, R - reset	Przechowuje 1 bit (Q). S=1 ustawi $Q = 1$, R=1 zeruje, S=R=0 trzyma stan. Dwa sprzężone NOR -y.
Sterowany poziomem	aktywny gdy CLK = 1	Wejścia S / R bramkowane AND-em z zegarem. Stan zmienia się tylko przy CLK=1 .
Sterowany zboczem	zbocze 1 $\rightarrow$ 0	Dwa przerzutniki poziomowe w kaskadzie (master-slave), drugi z zanegowanym zegarem. Stan przepisuje się w momencie zmiany zegara.

2. Typy danych

Typ	char	short	int	long	float	double	void*
x86-64	1	2	4	8	4	8	8

3. Kodowanie liczb i konwersja

$$\text{B2U}(X) = \sum_{i=0}^{w-1} x_i 2^i \quad \text{B2T}(X) = -x_{w-1} 2^{w-1} + \sum_{i=0}^{w-2} x_i 2^i$$

$$\text{T2U}(x) = x < 0 ? x + 2^w : x \quad \text{U2T}(u) = u > \text{TMax} ? u - 2^w : u$$

Skróty: **B** = bity, **U** = unsigned, **T** = ze znakiem (kod U2).
Stąd **B2U** = bity $\rightarrow$ unsigned, podobnie **U2T**, **UMax**, **TMax**, **UAdd**, **TAdd**.

Stała	Bity	Wartość
UMax	11...1	$2^w - 1$
TMax	01...1	$2^{w-1} - 1$ (INT_MAX)
TMin	10...0	-2^{w-1} (INT_MIN)
−1	11...1	te same bity co UMax

Promocja w C: **unsigned** i **int** w jednym wyrażeniu/porównaniu ($< > ==$) $\rightarrow$ **int** promowany do **unsigned** (bity bez zmian, $-1 \rightarrow \text{UMax}$).

4. Rozszerzanie, obcinanie i przesunięcia

Rozszerz. 0 (zext)	unsigned : dopisuje 0 z góry
Rozszerz. znak (sext)	signed : powiela bit znaku (MSB)
$x \ll k$	$x \cdot 2^k$ (sgn./uns.)
$u \gg k$	$\lfloor u/2^k \rfloor$ - logiczne (zera)
$x \gg k$	arytmetyczne (kopia MSB), zaokr. do $-\infty$
$x/2^k$ do 0	$(x + (1 \ll k) - 1) \gg k$
Strength reduction	$x * 24 \rightarrow (x \ll 5) - (x \ll 3)$

5. Operacje, overflow i endianness

UAdd/TAdd	$(u + v) \bmod 2^w$	
Nadmiar (+)	$u, v > 0$, a wynik < 0 (wynik -2^w)	
Nadmiar (-)	$u, v < 0$, a wynik ≥ 0 (wynik $+2^w$)	
Negacja U2	$\sim x = \sim x + 1$; $\sim \text{TMin} = \text{TMin}$, $\sim 0 = 0$	
Wykr. nadmiaru (s=x+y)	$((s \wedge x) \& (s \wedge y)) \gg (w-1)$	
Maska / abs	$m = x \gg 31$; $\text{abs} = (x \wedge m) - m$	
Schemat	Najniższy adres	0x0123
Big Endian	MSB	[01][23]
Little Endian	LSB	[23][01]

6. Zmiennoprzecinkowe (IEEE 754)

$$V = (-1)^s \times M \times 2^E \qquad \text{Bias} = 2^{k-1} - 1$$

Format	bity	s	exp (k)	frac (n)
float	32	1	8	23
double	64	1	11	52

Kategorie wartości

Kategoria	exp	Własności
Znormalizowane	$\neq 0 \dots 0$ $\neq 1 \dots 1$	$E = \text{exp} - \text{Bias}$. $M = 1.\text{frac}$. Najgęściej ułożone wokół 0.
Zdenormaliz.	$= 0 \dots 0$	$E = 1 - \text{Bias}$. $M = 0.\text{frac}$. Reprezentują ± 0.0 (frac = 0) i liczby bardzo bliskie zera.
Specjalne	$= 1 \dots 1$	$\text{frac} = 0 \rightarrow \pm \infty$ (nadmiar, dzielenie przez 0). $\text{frac} \neq 0 \rightarrow \text{NaN}$ (np. $\sqrt{-1}$, $\infty -$ ∞).

Zaokrąglanie GRS i Round-to-Even

Domyślny tryb to **Round-to-Even** (zapobiega błędowi statycznemu przy sumowaniu).



G - ostatni zachowany, **R** - pierwszy usunięty, **S** - OR reszty

Warunek	Ułamek	Akcja
R=0	< 0.5	W dół (odcięcie)
R=1, S=1	> 0.5	W górę (+1 do G)
R=1, S=0	$= 0.5$	Do parzystej: w górę gdy G=1 , w dół gdy G=0

Własności matematyczne i rzutowanie

Brak łączności	$(a + b) + c \neq a + (b + c)$ - zaokrąglenia
Brak rozdzielności	$a(b + c) \neq ab + ac$
int $\rightarrow$ float	możliwa utrata precyzji (zaokrągła)
int $\rightarrow$ double	bezstratne ($52 > 32$ bity)
float/double $\rightarrow$ int	obcina do 0; poza zakresem lub NaN $\rightarrow$ TMin (<code>0x80000000</code>)

7. Rejestry x86_64

%rax	%eax %ah %al	%rbx	%ebx %bh %bl
%rcx	%ecx %ch %cl	%rdx	%edx %dh %dl
%rsi	%esi %si %sil	%rdi	%edi %di %dil
%rbp	%ebp %bp %bpl	%rsp	%esp %spl
%r8	%r8d %r8w %r8b	%r9	%r9d %r9w %r9b

Uwaga: Rejestry od **%r10** do **%r15** używają schematu:
%r10, **%r10d**, **%r10w**, **%r10b**.

Podział ról rejestrów

%rax	Akumulator. Główny rejestr arytmetyczny. Przechowuje wartość zwracaną .
%rdi, %rsi, %rdx, %rcx, %r8, %r9	Kolejno: od 1. do 6. argumentu funkcji . Caller-saved.
%rsp	Wskaźnik stosu (SP). Wskazuje wierzchołek ramki.
%rbp	Wskaźnik bazy (BP). Początek ramki stosu. Callee-saved.
%rbx, %r12-%r15	Ogólnego przeznaczenia. Wszystkie callee-saved .
%rip	Wskaźnik instrukcji (Program Counter).

Rejestry wektorowe (zmiennoprzecinkowe)

%xmm0-%xmm15	128-bitowe. %xmm0 to wartość zwracana (float / double). Pierwsze 8 to argumenty. Caller-saved.
%ymm0-%ymm15	256-bitowe rozszerzenie XMM. Dzielą z nimi najmłodsze 128 bitów.

8. Tryby adresowania (operandy)

Tryb	Format	Obliczanie adresu
Natychnmiastowy (Imm)	\$Imm	Imm
Rejestrowy (Reg)	%Ra	%Ra
Bezpośredni (Mem)	Imm	M[Imm]
Pośredni (Mem)	C(%Rb)	M[C%Rb]
Z przesunięciem	D(C%Rb)	M[C%Rb + D]
Skalowany (Ind/scaled)	D(C%Rb, %Ri, S)	M[C%Rb+%Ri*S+D]
Skalowany (baseless)	(, %Ri, S)	M[C%Ri * S]

Legenda: **Imm** / **D** to stała liczbowa, **%Ra** / **%Rb** to rejestr bazowy, **%Ri** to rejestr indeksowy, **a** / **S** to skala (tylko: 1, 2, 4 lub 8).

9. Flagi stanu i sterowanie

ZF	Zero. Wynik to 0 (np. argumenty są równe).
SF	Sign. Wynik jest ujemny (MSB = 1).
CF	Carry. Przepełnienie dla liczb bez znaku (unsigned).
OF	Overflow. Przepełnienie dla liczb ze znakiem (signed).

Sufiksy warunkowe (dla `jX`, `setX`, `cmovX`)

Sufiks	Znaczenie
<code>e / z</code>	Equal / Zero (równe / wynik to 0)
<code>ne / nz</code>	Not Equal / Not Zero (nierówne)
<code>s</code>	Sign (wynik ujemny, <code>SF=1</code>)
Liczby ze znakiem (signed)	
<code>g / ge</code>	Greater / Greater or Equal
<code>l / le</code>	Less / Less or Equal
Liczby bez znaku (unsigned)	
<code>a / ae</code>	Above / Above or Equal (No Carry)
<code>b / be</code>	Below / Below or Equal (Carry)

Translacja struktur kontrolnych (C → ASM)

- **if-else:** `jX` (skok) lub `cmovX` (liczy obie ścieżki, bez kary za predykcję). `cmovX` odpada przy drogich gałęziach, wyłuskaniach (`*p`) i efektach ubocznych (`x++`).
- **switch:** tablica skoków → czas $O(1)$. Skok pośredni: `jmp *.L4(, %rdi, 8)`. Brak `break` ⇒ **fall-through**.
- **Pętla for:** zawsze redukowana do **while**:
`init; while(cond) { body; update; }`

Wzorce asemblerowe dla pętli:

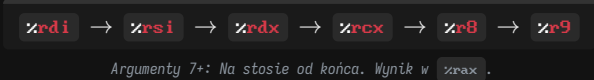
do-while	while (Jump-to-mid)	while (Guarded)
<code>loop:</code> Body if(T) goto loop	<code>goto test</code> loop: Body test: if(T) goto loop	<code>if(!T) goto done</code> loop: Body if(T) goto loop done:

10. Procedury i stos

Stos rośnie **w dół** (niższe adresy); `%rsp` wskazuje **wierzchołek** (najniższy zajęty adres). `push` zmniejsza `%rsp` o 8, `pop` zwiększa o 8.

- `call dest`: odkłada adres powrotu na stos i skacze do `dest`.
- `ret`: zdejmuje adres powrotu ze stosu i skacze pod niego.

11. Konwencja Wywoływań (System V ABI)



Rejestry caller-saved vs callee-saved

Caller-saved	Callee-saved
(Wołający musi zapisać) Mogą zostać nadpisane w funkcji. By je zachować, caller kładzie je na stos przed <code>call</code> .	(Wołany musi przywrócić) Muszą zachować stan. Callee musi zapisać je na stos i odtworzyć przed <code>ret</code> .
<code>%rax</code> , <code>%rdi</code> , <code>%rsi</code> , <code>%rdx</code> , <code>%rcx</code> , <code>%r8</code> - <code>%r11</code>	<code>%rbx</code> , <code>%rbp</code> , <code>%r12</code> - <code>%r15</code>

Ramka stosu

Każde wywołanie funkcji ma własną ramkę (stąd **rekurencja** działa naturalnie).

Ramka potrzebna, gdy: brak rejestrów na zmienne (spilling), lokalna tablica/struktura, lub pobrano adres zmiennej (`&`).

Prolog	Epilog
<code>pushq %rbp ;</code> zapisz stary base ptr <code>movq %rsp, %rbp ;</code> nowy base ptr = rsp <code>subq \$32, %rsp ;</code> alokacja 32 bajtów	<code>addq \$32, %rsp ;</code> zwolnij alokację <code>popq %rbp ;</code> przywróć base ptr <code>ret ;</code> skok powrotny
<code>leave</code> : zastępuje <code>movq %rbp, %rsp</code> + <code>popq %rbp</code> jedną instrukcją.	

12. Architektura CPU

Fazy przetwarzania instrukcji

Fetch → Decode → Execute → Memory → Write

Pipelining i hazardy

Nakładanie faz zwiększa throughput, ale rodzi konflikty (hazardy):

Danych	Odczyt po zapisie - wynik jeszcze nie jest w rejestrze, a kolejna instrukcja już go potrzebuje. Rozwiązanie: forwarding/bypassing (wynik z ALU prosto na wejście) lub stall/bubble.
Sterowania	Skok wymusza odgadnięcie następnego adresu. Rozwiązanie: branch prediction. Pomyłka → czyszczenie potoku (misprediction penalty).

Out-of-Order

- Nowoczesne procesory nie wykonują instrukcji sekwencyjnie:
- **Superskalarność:** wiele instrukcji na cykl (wiele jednostek wykonawczych).
 - **Register renaming:** mapowanie rejestrów logicznych (`%rax`) na wiele fizycznych - eliminuje fałszywe zależności.
 - **Reorder buffer:** instrukcje liczone asynchronicznie, ale zatwierdzane w kolejności programu (spójność).

13. Tablice i struktury

Reprezentacja tablic w pamięci

Typ	Deklaracja	Adres
Jednowymiarowa	<code>T A[1]</code>	$A[i] = x_A + i \times \text{sizeof}(T)$
Wielowymiarowa	<code>T A[R][C]</code>	$A[i][j] = x_A + (i \times C + j) \times \text{sizeof}(T)$
Wielopoziomowa	<code>T *A[1]</code>	$M[x_A + i \times 8] + j \times \text{sizeof}(T)$ (dwa odczyty z pamięci)

Wyrównanie danych i padding

- **Zasada ogólna:** obiekt rozmiaru K leży pod adresem podzielnym przez K .
- **Struktury:** każde pole wyrównane do własnego K ; rozmiar całości podzielny przez największe wyrównanie (padding na końcu).
- **Optymalizacja:** pola od największego do najmniejszego → minimalny padding.

14. Układ pamięci

Mapa pamięci procesu

Stack ↓	Rośnie w dół. Zmienne lokalne, adresy powrotu (limit 8MB).
Shared libs	Współdzielone biblioteki (np. <code>libc.so</code>).
Szeregi ↑	Rośnie w górę. Dynamiczna alokacja (<code>malloc</code> , <code>new</code>).
Data	Zmienne globalne i statyczne (zainicjowane i niezainicjowane).
Text	Read-only. Binarny kod wykonywalny programu.

Góra tabeli = wysokie adresy, dół = niskie adresy.

Zagrożenia i mechanizmy obronne

Buffer overflow	Brak sprawdzania granic tablic. Nadpisuje stos (stary <code>%rbp</code> , adres powrotu) → przejęcie sterowania.
ROP (Gadżety)	Return-Oriented Programming. Sklejanie legalnych skrawków kodu zakończonych <code>ret</code> - omija zakaz wykonywania (NX).
Stack canaries	Losowa wartość tuż przed adresem powrotu. Przed <code>ret</code> weryfikacja (<code>xor + je</code>); uszkodzona → <code>abort()</code> .
NX / ASLR	NX: zakaz wykonywania kodu ze stosu i sterty. ASLR: losowanie bazowych adresów obszarów przy każdym starcie.

Unie

Wszystkie pola współdzielą **ten sam adres** (offset 0); rozmiar = **największe pole**. Służą do manipulacji bitowych z ominięciem systemu typów (np. bity `float` jako `int`).

15. Optymalizacje i ich ograniczenia

Kompilator nie zoptymalizuje kodu, jeśli mogłoby to zmienić zachowanie programu (choćby dla specyficznych argumentów).

Optymalizacyjne blokady

Aliasing pamięci	<code>*p</code> i <code>*q</code> mogą wskazywać na to samo → wymuszony odczyt/zapis do RAM zamiast rejestru. Pomaga słowo kluczowe <code>restrict</code> .
Wywołania funkcji	Funkcja w pętli może mieć ukryte efekty uboczne - nie zostanie wyrzucona poza pętlę. Rozwiązanie: inlining lub makra.
Arytmetyka float	Brak łączności: $(a + b) + c \neq a + (b + c)$. Kompilator nigdy nie zmieni kolejności działań (troska o precyzję).

Podstawowe techniki optymalizacji

Code motion	Wyrzucenie niezmienników (np. <code>x * y</code>) poza pętlę.
Strength reduction	Kosztowne → tańsze (np. <code>x * 64</code> → <code>x << 6</code> , iteracja wskaźnikiem zamiast mnożenia indeksu).
Share subexpr.	Jednokrotne obliczanie powtarzających się podwyrażeń.
Loop unrolling	Rozwinięcie pętli (np. krok co 2). Mniejszy narzut pętli, więcej równoległości dla CPU.

16. Linkowanie i konsolidacja

Fazy budowania programu

cpp (Preprocesor) → cc1 (Kompilator) → as (Asembler) → ld (Linker)

Formaty plików obiektowych (ELF)

- Relokowalne (.o): kod i dane gotowe do połączenia z innymi .o .
- Wykonywalne (a.out): sklejone, gotowe do załadowania i uruchomienia.
- Współdzielone (.so): linkowane dynamicznie przy ładowaniu lub w trakcie działania.

.text	Skompilowany kod maszynowy.
.rodata	Dane tylko do odczytu (np. "Witaj" w printf , tablice switch).
.data	Zainicjowane zmienne globalne i statyczne (np. int x = 5;).
.bss	Niezainicjowane globalne/statyczne - nie zajmują miejsca w pliku.
.symtab	Tablica symboli (funkcje i zmienne globalne).
.rel.text / .data	Wpisy relokacji: gdzie i jak linker ma wstawić ostateczne adresy.

Symbol resolution

Linker rozróżnia 3 typy symboli:

- Globalne: zdefiniowane tu, używane tam,
- Zewnętrzne: extern - używane tu, zdefiniowane gdzie indziej,
- Lokalne: z C-owym słowem static .

Uwaga: Zmienne lokalne na stosie NIE są w ogóle widoczne dla linkera

Silne (Strong): Funkcje i zainicjowane zmienne globalne (np. int foo = 5;).
Słabe (Weak): Niezainicjowane zmienne globalne (np. int foo;).

- Reguła 1 Wiele silnych → błąd linkowania (Multiple definition).
- Reguła 2 1 silny + n słabych → wygrywa silny.
- Reguła 3 Wiele słabych → linker wybiera dowolny (gcc -fno-common zakazuje).

Relokacja

Linker skleja sekcje .text / .data z plików .o w jeden blok, nadaje finalne adresy (run-time) i poprawia wszystkie odniesienia w kodzie wg wpisów z .rel .

Biblioteki

Statyczne (.a)
Zbiór plików .o . Linker kopiuje tylko używane moduły.
Wady: każda aplikacja ma własną kopię w RAM, zmiana wymusza relink.
Flagi gcc : -a podaje się na końcu komendy.

Dynamiczne (.so)
Kod ładowany do pamięci raz; adresy dostarczane w trakcie działania (przez GOT).
Zalety: znacznie mniejsze binarki, łatwe aktualizacje.

17. Katalog instrukcji (AT&T)

Opcode	Efekt	Opis
mov S, D	$D \leftarrow S$	Kopiuje wartość z S do D.
lea S, D	$D \leftarrow \text{addr}(S)$	Oblicza adres S (bez czytania pamięci).
add S, D	$D \leftarrow D + S$	Dodawanie (ustawia flagi).
sub S, D	$D \leftarrow D - S$	Odejmowanie (ustawia flagi).
imul S, D	$D \leftarrow D * S$	Mnożenie liczb ze znakiem.
sal / shl k, D	$D \ll = k$	Przesunięcie w lewo (mnożenie przez 2^k).
sar k, D	$D \gg = k$	Arytmetyczne w prawo (dzielenie). Kopiuje znak.
shr k, D	$D \gg = k$	Logiczne w prawo. Dopełnia zerami.
and / or S, D	$D \leftarrow D \& / S$	Operacje bitowe (ustawiają flagi).
xor S, D	$D \leftarrow D \wedge S$	Często jako xor %rax, %rax do zerowania.

Porównania i sterowanie

cmp S1, S2	$S2 - S1$	Ustawia flagi jak odejmowanie, nie zapisuje wyniku.
test S1, S2	$S2 \& S1$	Ustawia flagi jak AND.
jX dest	$\text{if}(X) \%rip \leftarrow \text{dest}$	Skok warunkowy (X = warunek).
setX D	$\text{if}(X) D \leftarrow 1$	Ustawia najmłodszy bajt na 0 lub 1 wg flag.
cmovX S, D	$\text{if}(X) D \leftarrow S$	Warunkowe kopiowanie (zamiast skoku).

Stos i zarządzanie procedurami

push S	$\%rsp - = 8$ $M[\%rsp] \leftarrow S$	Odkłada na stos (zmniejsza %rsp).
pop D	$D \leftarrow M[\%rsp]$ $\%rsp + = 8$	Zdejmuje ze stosu do D (zwiększa %rsp).
call dest	push %rip $\%rip \leftarrow \text{dest}$	Skok do funkcji (zapisuje adres powrotu na stosie).
ret	pop %rip	Zdejmuje adres powrotu ze stosu i skacze pod niego.
leave	$\%rsp \leftarrow \%rbp$ pop %rbp	Sprząta ramkę stosu (odwrotność prologu).

Operacje wektorowe

Rozszerzenie AVX. Przedrostek `v` (nie niszczy źródeł).

<code>vmovaps / ups S, D</code>	$D \leftarrow S$	Kopiuje wektor. <code>a</code> (aligned - szybkie, adres podzielny przez wyrównanie), <code>u</code> (unaligned).
<code>vaddps / pd S1, S2, D</code>	$D \leftarrow S2 + S1$	Dodawanie wielokrotne (packed). <code>ps</code> (single-float), <code>pd</code> (double).
<code>vmulps / pd S1, S2, D</code>	$D \leftarrow S2 * S1$	Mnożenie wektorowe.
<code>vfmadd231ps S1, S2, D</code>	$D \leftarrow S2 * S1 + D$	Fused Multiply-Add. Mnoży i dodaje do <code>D</code> w jednym kroku.

Operacje zmiennoprzecinkowe

<code>movsd / movss S, D</code>	$D \leftarrow S$	Kopiuje skalar (<code>double</code> / <code>float</code>) między rejestrami XMM lub pamięcią.
<code>movapd / movaps S, D</code>	$D \leftarrow S$	Kopiuje wyrównany wektor zmiennoprzecinkowy.
<code>addsd / subsd S, D</code>	$D \leftarrow D \text{ op } S$	Skalarne dodawanie / odejmowanie podwójnej precyzji.
<code>xorpd / xorps S, D</code>	$D \leftarrow D \hat{~} S$	Bitowy XOR na XMM. Jako <code>xorpd %xmm0, %xmm0</code> zeruje rejestr.
<code>ucomisd S1, S2</code>	$S2 - S1$	Porównuje <code>double</code> i ustawia flagi <code>ZF, PF, CF</code> w <code>%rflags</code> .

Uwaga o sufiksach: Instrukcje przyjmują sufiks określający rozmiar danych: `b` (8-bit), `w` (16-bit), `l` (32-bit), `q` (64-bit).
Np. `movl` kopiuje 32 bity (i automatycznie zeruje górną połowę 64-bitowego rejestru!).